



## TRABAJO PRÁCTICO

### Introducción y Objetivo General

El videoclub “Cinefilia” se encuentra en una etapa de transición digital. El objetivo es desarrollar un sistema de gestión integral en **lenguaje C** que no solo administre el catálogo físico actual, sino que también actúe como motor de **Business Intelligence (BI)** para identificar qué socios son aptos para migrar a una futura plataforma de streaming. :

- Gestionar miembros y títulos de películas.
- Validar información proveniente de archivos externos.
- Administrar alquileres y generar estadísticas.
- Implementar estructuras en memoria.
- Aplicar algoritmos de búsqueda, ordenamiento y validación genérica.

### Carga y Validación de datos

Archivo .csv, con campos separados por punto y coma(,)

#### Miembros

Cada registro representa un miembro del videoclub y contiene los siguientes campos:

| Campo                      | Tipo      | Validación                                                                                                                                                                                                      |
|----------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DNI (clave)                | long      | 1000000 < DNI <100000000.                                                                                                                                                                                       |
| CUIL                       | char[]    | Calculado a partir del DNI (ver punto 1-)                                                                                                                                                                       |
| Apellidos y Nombres        | char (60) | Ver definición (ver punto 2-)                                                                                                                                                                                   |
| Fecha de Nacimiento        | t_fecha   | Fecha válida y anterior a la fecha de proceso en al menos 10 años                                                                                                                                               |
| Sexo                       | char      | 'F' (Femenino), 'M' (Masculino), 'O' (Otro)                                                                                                                                                                     |
| Fecha de afiliación        | t_fecha   | Válida, => fecha de nacimiento y <= fecha de proceso                                                                                                                                                            |
| Categoría                  | char (10) | La categoría de obtendrá del campo fecha de nacimiento;<br>'MENOR', si es menor de 18 años.<br>'ADULTO', si es mayor de 18 años.<br>Si el miembro es MENOR de edad, deberá tener un el mail del tutor asociado. |
| Fecha de última cuota paga | t_fecha   | => Fecha de afiliación y <= fecha de proceso.                                                                                                                                                                   |
| Estado                     | char      | 'A' Alta, 'B' Baja - Inicialmente 'A'.                                                                                                                                                                          |
| Plan                       | Char (10) | 'BASIC', 'PREMIUM', 'VIP', 'FAMILY'.<br>Se permitirá elegir las opciones de un menú si se da de alta un miembro.                                                                                                |
| Email Tutor                | Char (30) | Si el miembro es mejor de edad, este campo no puede estar vacío.<br>Ver definición 3-                                                                                                                           |



## 1-CUIL

### Qué es el CUIL/T?

El CUIL/T es el **Código Único de Identificación Laboral – Código Único de Identificación Tributaria**. El mismo consta de 11 (once) números. Los 10 (diez) primeros (2 + 8) conforman el **Código de Identificación** y el último conforma el **Dígito de Verificación**.

Para obtener los elementos mencionados en el párrafo anterior se aplica el siguiente algoritmo matemático:

**XY – 12345678 – Z**

**XY:** Indican el Tipo (Hombre, Mujer, Sociedad o Empresa)

**12345678:** Numero de DNI

**Z:** Dígito de Verificación

### Se determina XY de la siguiente manera:

Hombre = 20

Mujer = 27

Empresa o Sociedad = 30

### Se multiplica XY 12345678 por un número de forma separada:

Dado XY = 20, a modo de ejemplo.

$$2 * 5 = 10$$

$$0 * 4 = 0$$

$$1 * 3 = 3$$

$$2 * 2 = 4$$

$$3 * 7 = 21$$

$$4 * 6 = 24$$

$$5 * 5 = 25$$

$$6 * 4 = 24$$

$$7 * 3 = 21$$

$$8 * 2 = 16$$

**Ahora se suman los resultados de las multiplicaciones como se muestra a continuación:**

$$10 + 0 + 3 + 4 + 21 + 24 + 25 + 24 + 21 + 16 = 148$$

**El resultado calculado en el paso anterior se divide por 11 (once) y se obtiene el resto de dicha división.**

$$148 / 11 = 13 \text{ (División Entera)}$$

$$\text{Resto: } 148 - (13 * 11) = 5$$

**Una vez determinado el resto se aplican las siguientes reglas:**

Si el resto es igual a 0 (cero), entonces Z (Dígito de Verificación) es igual a 0 (cero).

Si el resto es igual a 1 (uno) ocurre lo siguiente:

- Si es Hombre, entonces Z (Dígito de Verificación) es igual a 9 (nueve) y XY es igual a 23 (veintitrés).

- Si es Mujer, entonces Z (Dígito de Verificación) es igual a 4 (cuatro) y XY es igual a 23 (veintitrés).

- En cualquier otro caso Z (Dígito de Verificación) es igual a 11 (once) menos el resto del



cociente.

Resto = 5

$11 - 5 = 6$

Z = 6

**Resultante CUIL: 20 – 12345678 – 6**

## 2-Normalización

Los apellidos y nombres deberán normalizarse de la siguiente forma:

Deben comenzar con letra mayúscula y luego continuar con minúscula.

- Deben estar separado/s del/los nombre/s por una coma. De no existir dicha coma, agregarla a continuación de la primera palabra.
- Cada palabra debe separarse por no más de un espacio.

**ejemplo: galletA pePE raul** → Galleta, Pepe Raul

Todas las funciones de validación deberán ser funciones genéricas con parámetros que contengan punteros a funciones. [\\*A consultar con la profe](#)

## 3-Validación cuenta de mail

**usuario@dominio.com.[opcional].**

Se compone de tres partes esenciales: el nombre de usuario (identificador), el símbolo arroba (@) y el dominio (servidor de correo). Debe ser único, profesional y sin espacios, permitiendo letras, números y puntos. [] opcional, por lo general corresponde al país de origen, ejemplo .ar

**Estructura detallada:**

- **Parte local (Usuario):** Identifica al usuario, por ejemplo: juan.perez.
- **Símbolo arroba (@):** Separa el usuario del dominio. Es obligatorio y único.
- **Dominio:** El proveedor del servicio
- **[opcional]:** País origen  
(ej. gmail.com, outlook.com, empresa.com, yahoo.com.ar)

Además de los miembros, el sistema debe gestionar el catálogo de películas del videoclub.

## Títulos:

Cada registro representa un título del videoclub y contiene los siguientes campos:

| Campo               | Tipo     | Validación                                       |
|---------------------|----------|--------------------------------------------------|
| ID Película (Clave) | int      | Autoincremental o único.                         |
| Título              | char(60) | Normalizado (ídem nombres de miembros).          |
| Género              | char(20) | 'Acción', 'Drama', 'Comedia', 'Terror'           |
| Stock               | int      | Cantidad de VHS disponibles. >=1, 0 por defecto. |



### **Primera Parte:**

El sistema deberá validar por cada archivo de entrada cada registro. Ante el primer campo del registro que de error, el mismo quedará afuera y se deberá guardar como un error dentro de; una estructura de doble entrada, donde:

### **Auditoría de Errores de miembros**

Compuesta por :

- el tipo de validación fallida (DNI, Fecha, email, etc.)
- la cantidad de incidencias
- y el DNI asociado al error.

### **Auditoría de Errores de títulos**

Compuesta por :

- el tipo de validación fallida (ID, Género y stock)
- la cantidad de incidencias
- y el ID de película asociado al error.

### **Aplicación**

El sistema solicitará la fecha de proceso y la validará.

Fecha de proceso: si no se ingresa alguna, tomará la actual del sistema, sino deberá ser una válida.

Luego, se leerán las datos de miembros y títulos, realizando las distintas validaciones y generando con los no válidos la estructura solicitada.

## **Segunda parte**

### **Menú de Operaciones**

El menú debe incluir las siguientes funcionalidades :

- a. Alta de miembro
- b. Alta de un título
- c. Baja de un miembro
- d. Baja de un título
- e. Modificación de un miembro
- f. Modificación de un título
- g. Mostrar información de un miembro
- h. Alquiler de un título
- i. Listado de miembros ordenados por DNI
- j. Listado miembros por Plan
- k. Salir

***Para esto deberá valerse del uso de un TDA Índice implementado en un array con asignación dinámica de memoria.***



**UNIVERSIDAD NACIONAL DE LA MATANZA**  
**Departamento de Ingeniería e Investigaciones Tecnológicas**  
**Tópicos de Programación**

**Alta títulos/miembro:** se obtendrán los datos del teclado, ingresando primero el DNI verificando que no exista en el índice. Una vez ingresados todos los datos del miembro, realizar la validación y consistencia de estos (ídem proceso de generación del archivo). Insertar en forma ordenada en el índice. Si se detectan errores, se ignora todo lo ingresado.

**Baja títulos/miembro:** Si existe la clave que se quiere dar de baja en el índice, actualizar el registro correspondiente con un carácter 'B' en estado. Eliminar registro correspondiente del índice.

**Modificación títulos/miembro:** Si existe la clave en el índice buscada, actualizar la información deseada. Si se detectan errores, se ignora todo lo ingresado.

**Mostrar información de un miembro/título:** Si existe la clave en el índice, mostrar toda la información correspondiente al socio.

**Alquiler de un título: *Relación Miembro-Título (Alquileres)***

Para gestionar qué miembro tiene qué película, el sistema deberá implementar una estructura de Alquileres Históricos en memoria, en la que se guardará la cantidad de veces que el miembro alquiló la película.

**Aclaración:** Los miembros con Plan BASIC no pueden alquilar más de 2 películas simultáneamente.

**Listado de socios ordenados por clave (DNI):** Debe mostrar todos los registros activos (no están dados de baja) ordenados por DNI.

**Listado miembros por Plan:** Debe mostrar el siguiente formato:

| Plan / Índice | (BASIC) | (PREMIUM) | (VIP) | (FAMILY) |
|---------------|---------|-----------|-------|----------|
| Aguirre, Aldo | DNI     | 0         | 0     | 0        |
| Argente, Pepe | 0       | DNI       | 0     | 0        |

Ordenado por Apellido y Nombre

**Salir:** Debe terminar la ejecución del programa.

Como procedimiento final, debe liberar la memoria del/los índice/s y grabar la información de los miembros, títulos y alquileres para persistir en disco con la fecha como parte del nombre, como también las auditorías. Si al ingresar nuevamente a la aplicación la fecha de proceso coincidiera con el nombre de los distintos archivos, se cargará estos datos en vez de los .csv originales.



### TDA Índice

Construir un TDA índice en los archivos indice.h e indice.c.

El desarrollo de ambos debe respetar la siguiente estructura:

```
///indice.h

#ifndef INDICE_H_INCLUDED
#define INDICE_H_INCLUDED

#define CANTIDAD_ELEMENTOS 100
#define INCREMENTO 1.3
#define OK 1
#define ERROR 0
#define NO_EXISTE -1

typedef struct
{
    unsigned nro_reg;
    long dni;
}t_reg_indice;

typedef struct
{
    void *vindice;
    unsigned cantidad_elementos_actual;
    unsigned cantidad_elementos_maxima;
}t_indice;

/*****
Descripción:    toma memoria para 100 elementos e inicializa la estructura va-
cía.
Parámetros:    indice: TDA índice.
                nmemb: cantidad de elementos del índice.
                tamanyo: el espacio en bytes ocupado por cada elemento.
Retorno:       n/a.
Observaciones:
*****/
void indice_crear(t_indice *indice, size_t nmemb, size_t tamanyo);
```



```
/******  
Descripción: redimensiona el tamaño del índice.  
Parámetros: índice: TDA índice.  
             nmemb: cantidad de elementos del índice.  
             tamaño: el espacio en bytes ocupado por cada elemento.  
Retorno: n/a.  
Observaciones: Debe proporcionar el nmemb incrementado en un 30%  
*****/  
void indice_redimensionar(t_indice *indice, size_t nmemb, size_t tamaño);  
  
/******  
Descripción: inserta en orden según la clave.  
Parámetros: índice: TDA índice.  
             registro: el nuevo elemento a insertar en el índice.  
             tamaño: el espacio en bytes ocupado por el elemento a insertar.  
             cmp: función de comparación provista.  
Retorno: OK si la operación fue exitosa y ERROR en caso contrario.  
Observaciones: Si el array está lleno, toma un 30 % más de memoria.  
*****/  
int indice_insertar (t_indice *indice, const void *registro, size_t tamaño,  
int (*cmp)(const void *, const void *));  
  
/******  
Descripción: elimina el registro del índice.  
Parámetros: índice: TDA índice.  
             registro: el elemento a eliminar.  
             tamaño: el espacio en bytes ocupado por el elemento a insertar.  
             cmp: función de comparación provista.  
Retorno: OK si la operación fue exitosa y ERROR en caso contrario.  
Observaciones: -  
*****/  
int indice_eliminar(t_indice *indice, const void *registro, size_t tamaño, int  
(*cmp)(const void *, const void *));
```



**UNIVERSIDAD NACIONAL DE LA MATANZA**  
**Departamento de Ingeniería e Investigaciones Tecnológicas**  
**Tópicos de Programación**

```
/******  
Descripción:      si la clave existe deja el registro en registro.  
Parámetros:      indice: TDA índice.  
                  registro: el elemento a buscar.  
                  nmemb: cantidad de elementos del índice.  
                  tamaño: espacio en bytes ocupado por el elemento a insertar.  
                  cmp: función de comparación provista.  
Retorno:         NO_EXISTE si no existe o si existe, la posición ocupada dentro  
del array.  
Observaciones:   -  
*****/  
int indice_buscar (const t_indice *indice, const void *registro, size_t nmemb,  
size_t tamaño, int (*cmp)(const void *, const void *));  
  
/******  
Descripción:      determina si el índice contiene 0 (cero) elementos.  
Parámetros:      indice: TDA índice.  
Retorno:         OK si está vacío, cualquier otro valor si no lo está.  
Observaciones:   -  
*****/  
int indice_vacio(const t_indice *indice);  
  
/******  
descripción:      determina si el índice contiene el tamaño máximo posible.  
Parámetros:      indice: TDA índice.  
Retorno:         OK si está lleno, cualquier otro valor si no lo está.  
Observaciones:   -  
*****/  
int indice_lleno(const t_indice *indice);  
  
/******  
Descripción:      deja el índice vacío.  
Parámetros:      indice: TDA índice.  
Retorno:         No posee.  
Observaciones:   -  
*****/  
void indice_vaciar(t_indice* indice);
```





**UNIVERSIDAD NACIONAL DE LA MATANZA**  
**Departamento de Ingeniería e Investigaciones Tecnológicas**  
**Tópicos de Programación**

```

/*****
Descripción:      Carga el array desde un archivo ordenado.
Parámetros:      path: la ruta al archivo binario.
                  indice: TDA índice.
                  vreg_ind: vector de elementos dentro del índice.
                  tamaño: el espacio en bytes ocupado por el elemento a inser-
tar.
                  cmp: función de comparación provista.
Retorno:         OK si la operación fue exitosa y ERROR en caso contrario.
Observaciones:   -
*****/
int indice_cargar(const char* path, t_indice* indice, void *vreg_ind, size_t
tamaño, int (*cmp)(const void *, const void *));

#endif // INDICE_H_INCLUDED

```



**Condiciones generales a cumplir:**

**I. Modularización y eficiencia según lineamientos de la cátedra:**

- A. **Bibliotecas**
- B. **Macros**
- C. **Estilo definido al nombrar variables claras**
- D. **Identación,**

según considere necesario

**II. Condiciones para la presentación**

- A. Presentar en tiempo y forma.
- B. Plataforma miel, indicando integrantes del grupo y Número
- C. El entregable deberá consistir en una url de Gitlab. No se acepta el código comprimido.
- D. La hora tope será el día establecido de presentación y registrada en la plataforma.
- E. Un solo integrante lo enviará a todos los docentes de la materia.
- F. **NO se permite trabajar solos**, SI NO tienen grupo se deberá avisar a los docentes por miel/teams.

**III. Devolución por parte docente**

- A. Se enviará por miel al grupo.
  - a. La nota será :
    - i. aprobado
    - ó
    - ii. se indicará qué se debe cambiar/solucionar.

Si no reciben en forma individual la nota vía miel, se deberá avisar a los docentes. NO importa si su grupo esta aprobado, la aprobación del tp es informada en forma individual por miel.

**IV. Evaluación del Trabajo Práctico.**

- A. **Una vez aprobado el trabajo práctico en forma grupal, se tomará en forma individual defensa de este en máquina, en forma presencial con una fecha establecida por la materia.**
- B. La defensa individual del trabajo práctico es condición **requerida** para la aprobación/cursada de la materia.
- C. La nota de la defensa individual será promediada con la del parcial.